📘 React Query (TanStack Query)

> **React Query** (now called **TanStack Query**) is a powerful library that helps us **fetch, cache, synchronize, and update server data** in React applications.

Instead of manually managing:

- `useState`
- `useEffect`
- Loading State
- Error State
- Refetching
- Caching

React Query handles all of these automatically, making your code cleaner and more efficient.

🤔 Why Do We Need React Query?

Normally, data fetching looks like this:

```
useEffect(() => {fetchUsers();
}, []);
```

With this approach, you need to manually manage:

- Loading state
- Error handling
- Success state
- Refreshing data

- Duplicate API calls
- Caching
- Retry logic
- Background refetching

As your application grows, this becomes difficult to maintain.

✅ **React Query solves all these problems with minimal code.**

🚀 Benefits of React Query

- ✅ Automatic API Fetching
- ✅ Built-in Caching
- ✅ Background Refetching
- ✅ Automatic Retry
- ✅ Pagination Support
- ✅ Infinite Scroll Support
- ✅ DevTools Support
- ✅ Request Deuplication
- ✅ Optimistic Updates
- ✅ Mutation Support (`POST`, `PUT`, `PATCH`, `DELETE`)

📦 Installation

Install React Query

```
npm install @tanstack/react-query
```

or

```
yarn add @tanstack/react-query
```

(Optional) Install DevTools

```
npm install @tanstack/react-query-devtools
```

📁 Project Structure

```
src
│
├── main.jsx
├── App.jsx
├── api
│   └── userApi.js
└── components
    └── Users.jsx
```

⚙️ Step 1: Create Query Client

File: `main.jsx`

```jsx
import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App";
import {QueryClient,QueryClientProvider} from "@tanstack/reac
t-query";
const queryClient=new QueryClient();

ReactDOM.createRoot(document.getElementById("root")).render(
<QueryClientProvider client={queryClient}>
<App/>
```

```
</QueryClientProvider>
);
```

> 💡 **What is `QueryClient`?**
>
> `QueryClient` is the central manager of React Query. It stores all cached data, controls refetching, retries, and manages every query in your application.

🌐 Step 2: Create an API Function

File: `api/userApi.js`

```
import axios from "axios";

export const fetchUsers = async () => {
const response = await axios.get("https://jsonplaceholder.typ
icode.com/users");
return response.data;
};
```

> 💡 Keep API functions separate from your components for better organization and reusability.

🔍 Step 3: Fetch Data using `useQuery`

```
import {useQuery } from"@tanstack/react-query";
import {fetchUsers } from"./api/userApi";

function Users() {
const { data, isLoading, error }=useQuery({
    queryKey: ["users"],
    queryFn:fetchUsers,
});
```

```
if (isLoading) return <h2>Loading...</h2>;
if (error) return <h2>Error...</h2>;
return (<>{data.map((user) => (<h3key={user.id}>{user.name}</
h3>))}</>);
}
```

✅ No need for `useEffect` or `useState` for fetching server data.

🏷️ Understanding `queryKey`

```
queryKey: ["users"]
```

`queryKey` is the **unique identifier** for your query.

React Query uses it to:

- Cache data
- Refetch data
- Invalidate data
- Share cached data across components

Examples

```
["users"]

["posts"]

["products"]

["user",10]

["posts",page]
```

> 💡 Different `queryKey`s create different cache entries.

⚡ Understanding `queryFn`

```
queryFn:fetchUsers
```

`queryFn` is the function responsible for fetching data.

Example:

```
const fetchUsers = async () => {
    const res = await axios.get("/users");
    return res.data;
};
```

Whenever React Query needs fresh data, it executes this function.

💾 What is Caching?

Imagine this scenario:

```
Home Page
     |
     ▼
Fetch Users
     |
     ▼
API Called ✅
     |
     ▼
Data Stored in Cache
```

Now the user navigates to another page and comes back.

❌ Without React Query

```
Visit Again

↓

API Called Again
```

✅ With React Query

```
Visit Again

↓

Data Loaded from Cache

↓

No Extra API Call
```

This makes your application much faster and reduces unnecessary network requests.

🔄 How Caching Works

```
First Visit

↓

API Call

↓

Store Data in Cache

↓

Open Again

↓

Read Cached Data

↓
```

```
Background Refetch (Optional)

↓

Update UI Automatically
```

This strategy is known as **Stale-While-Revalidate**.

📌 Example

```
const { data } = useQuery({
  queryKey: ["users"],
  queryFn : fetchUsers,
});
```

First Time

```
API Hit
```

Second Time

```
Cached Data Used
```

⏱️ `staleTime` vs `gcTime`

🟢 `staleTime`

Defines **how long the data is considered fresh.**

```
useQuery({
  queryKey: ["users"],
  queryFn:fetchUsers,
```

```
  staleTime:1000*60*5,
});
```

Meaning

- Data remains fresh for **5 minutes**
- During this period, React Query uses cached data
- No automatic refetch occurs while data is fresh

🟡 `gcTime` (Previously `cacheTime`)

Defines **how long inactive cached data stays in memory.**

```
useQuery({
  queryKey: ["users"],
  queryFn:fetchUsers,
  gcTime:1000*60*10,
});
```

Meaning

- If no component is using the query,
- React Query keeps it in memory for **10 minutes**
- After that, it removes the cache

🔄 Background Refetching

Even when cached data is displayed instantly, React Query can silently fetch fresh data in the background.

```
Show Cached Data

↓

Send API Request
```

```
↓

Receive Updated Data

↓

Update UI Automatically
```

The user experiences a fast UI while still receiving the latest data.

🔁 Automatic Retry

If an API request fails:

```
useQuery({
  queryKey: ["users"],
  queryFn:fetchUsers,
  retry:3,
});
```

React Query retries the request **up to 3 times** before showing an error.

🪟 Refetch on Window Focus

By default, React Query refetches **stale data** whenever the user returns to the browser tab.

Disable it like this:

```
useQuery({
  queryKey: ["users"],
  queryFn:fetchUsers,
  refetchOnWindowFocus:false,
});
```

🔄 Manual Refetch

```
const { data, refetch } = useQuery({
  queryKey: ["users"],
  queryFn:fetchUsers,
});

<button onClick={refetch}>Refresh</button>
```

Calling `refetch()` manually sends a new API request.

🛠️ React Query DevTools

Install DevTools:

```
npm install @tanstack/react-query-devtools
```

Add it to your application:

```
import { ReactQueryDevtools } from "@tanstack/react-query-dev
tools";

<QueryClientProvider client={queryClient}>
<App/>
<ReactQueryDevtools initialIsOpen={false}/>
</QueryClientProvider>
```

DevTools Help You Inspect

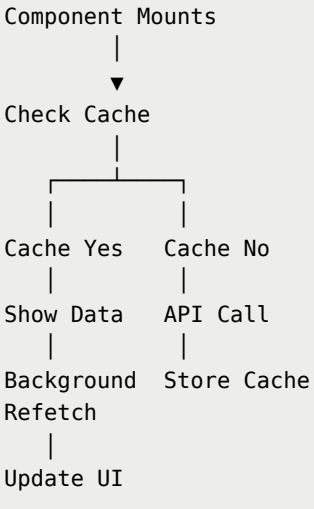
- Cached Queries
- Query Keys
- Query Status
- Fresh / Stale State

- Refetch Activity

⚙️ Common Query Options

```
useQuery({
  queryKey: ["users"],
  queryFn:fetchUsers,

  staleTime:1000*60*5,

  gcTime:1000*60*10,

  retry:3,

  refetchOnWindowFocus:false,

  enabled:true,
});
```

🔄 React Query Workflow



📌 When Should You Use React Query?

Use React Query whenever you're working with **server state**, such as:

- Users
- Products
- Posts
- Orders
- Comments
- Dashboard Data
- Search Results
- Pagination
- Infinite Scrolling
- Frequently Updated APIs

❌ When Not to Use React Query

React Query is **not** designed for client-side UI state.

Examples:

- Theme (Dark / Light)
- Sidebar Toggle
- Modal Open/Close
- Form Inputs
- Selected Tabs
- Local Counters

For these, use:

- `useState`

- Context API
- Redux
- Zustand

📝 Summary

React Query (TanStack Query) is a powerful library for managing **server state** in React applications. It automatically handles API requests, caching, loading and error states, retries failed requests, background synchronization, and data updates. By reducing boilerplate code and improving performance with intelligent caching, it helps developers build faster, cleaner, and more scalable applications.